

A Comprehensive Analysis of Mahalanobis Distance Effectiveness for Out-of-Distribution Data Detection

Thijmen Adriaansen^[2220822], Filip Atanasovski^[1833103], Matus Feltovic^[1718657],
and Thijmen Schinkelshoek^[1663305]

Eindhoven University of Technology

Abstract. We investigate out-of-distribution (OOD) detection using Mahalanobis distance feature-space-based scoring compared to Maximum Softmax Probability (MSP) based scoring across multiple datasets, model layers, and OOD settings (Far-OOD, Near-OOD, etc.). Using fine-tuned MobileNet V2/V1 models we calculate layer-wise Gaussian feature models, which we use to calculate Mahalanobis distance and evaluate the effectiveness of differentiation of OOD from in-distribution (ID) in different layers, as well as investigate whether combining different layer feature models increases the differentiation effectiveness of the Mahalanobis distance based OOD detection. Results show that both Mahalanobis distance and MSP perform almost identically in far-OOD settings. However, in near-OOD, MSP is slightly more effective for the highly accurate classifiers, while Mahalanobis becomes clearly superior in the lower-accuracy and lower-confidence classifier.

Keywords: Mahalanobis distance · Maximum Softmax Probability · MobileNet.

1 Introduction

When deep neural network classifiers are deployed, they are supposed to receive inputs from the same distribution as the data they were trained on. This is known as a closed-world assumption. This assumption is seldom true in real life though. When a model receives an out-of-distribution (OOD) input, that is an input that is different from anything observed during training, a deployed model still must classify the input into its known classes. Although the model has no class for such an input, the model is still required to assign one. Even worse, it typically does so with a high degree of confidence [9] which provides no indication that the input was actually unusual. By determining whether an input is in-distribution (ID) prior to its prediction being trusted, OOD detection provides this missing signal.

Two concerns that arise from a model’s quiet overconfidence serve as the driving force behind this effort. The first is safety: where an autonomous car acting on a sign it has never seen is an example of how a confident but incorrect

prediction on an unknown input, may lead to a dangerous downstream decision [13]. Security is the second and most important issue: An adversary with just black-box query access tries to flood the model with inputs in a model-extraction (or cloning) assault. Then they log the predictions that are returned and train a copy on the resultant input-prediction pairs [12]. These queries are thus predominantly OOD as the attacker rarely has access to the model’s training data [10, 4]. Because it may identify a cloning effort from the very inputs that enable the assault, a detector that flags OOD inputs also serves as a defense to this attack.

This issue is addressed by two kinds of detectors. Confidence-based approaches are inexpensive, but inherit the network’s overconfidence and are readily tricked [9]. They read the classifier’s own output: most simply through the maximal softmax probability (MSP) [3]. Instead, feature-based approaches scan the network’s intermediate representations, and where inputs belonging to the same class cluster together. The Mahalanobis-distance detector of Lee et al. [5] flags inputs that are far from any cluster by fitting one Gaussian per class to these features and scoring a fresh input based on its distance to the closest class cluster. It is said to perform better than confidence-based baselines by reading the geometry of the feature space rather than a single confidence value.

Any of the layers of the network can compute the Mahalanobis detection. Therefore the resultant per-layer scores can be aggregated into a single score. This then raises two useful questions which are: does combining layers outperform just choosing the best single layer and which layer best distinguishes the ID from the OOD inputs?

In this paper we present a thorough analysis of the Mahalanobis-distance detector along various axes. This is done in three phases of increasing difficulty: a far-ODD setting (with traffic signs versus ImageNet), then a near-ODD setting (with traffic signs versus a different set of held-out traffic-sign classes), and lastly a more difficult in-distribution setting where the classifier itself is less powerful (fine-grained pet breeds), where we refine MobileNetV2 classifiers and assess the detector at each block. We benchmark against the MSP baseline and compare single layers against three different layer-combination schemes, and repeat the study on a second architecture (MobileNetV1) to see if the results could be generalized.

2 Background and Related Work

This section reviews the prior work our study is built on. We first describe OOD inputs and the risks they create in Section 2.1. We then review the two families of detection methods in the literature: one that reads the classifier’s output confidence (Section 2.2), and one that reads the features inside the network (Section 2.3). The second family contains the Mahalanobis-distance detector that this project evaluates. We end with the security setting that motivates the work in Section 2.4.

2.1 Out-of-distribution inputs and their risks

A classifier model learns from a fixed set of training data and is then applied to never-seen inputs. An input that resembles the training data is called in-distribution (ID), and an input that differs substantially from it is called out-of-distribution (OOD). Clearly, a classifier has no class for an OOD input, yet it would still assign one of its known classes. In that case, the prediction would be unreliable, even though the model gives no signal that the input was unfamiliar [3, 13]. OOD detection adds this signal by deciding whether an input is in-distribution before its prediction is trusted.

This matters for two reasons. The first is safety. A model deployed in the real world (e.g., in a self-driving car) can receive inputs unlike anything in its training data, in which case a confident but wrong prediction can lead to an unsafe decision [13]. The second is security. An attacker can use unfamiliar inputs to copy a model through its query interface. The safety risk applies to the detection methods below, while the security risk is discussed on its own in Section 2.4.

2.2 Confidence-based detection

The most direct way to judge an input is to read the classifier’s own confidence score. A neural network classifier ends in a softmax layer, which turns the network’s raw outputs into a probability $P(y = c \mid x)$ for each known class c . The maximum softmax probability (MSP) is the largest of these probabilities:

$$\text{MSP}(x) = \max_c P(y = c \mid x). \quad (1)$$

A high value means the network is confident that the input belongs to a known class, and a low value suggests an unfamiliar input [3]. This score is the standard baseline for out-of-distribution detection.

However, on its own the score is unreliable, because neural networks are over-confident. The softmax uses an exponential, so small changes in the network’s output produce large and sharp probabilities [3]. A network, therefore, often reports high confidence on inputs it has never seen. Nguyen et al. produce images that are unrecognizable to humans (e.g., random noise) yet are still labeled as familiar objects with over 99% confidence [9]. Modern networks are also poorly calibrated, meaning their reported confidence does not match how often they are correct [2]. The maximum softmax probability still works as a baseline, but it is the weakest method we compare against.

Other methods exist that improve the separability of the softmax score without retraining the network. For example, ODIN scales the outputs by a temperature and adds a small perturbation to the input, which widens the gap between in- and out-of-distribution scores [6]. It has more accurate detection compared to the standard softmax baseline, but it tunes its settings on OOD examples [5], which are not available when the unfamiliar inputs cannot be known in advance. Both the baseline and ODIN read only the network’s final output, so they cannot use the richer information inside the network.

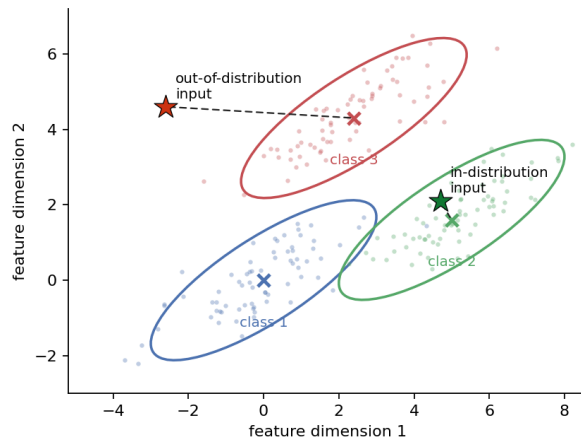


Fig. 1. In-distribution inputs form one cluster per class in the network’s feature space, shown here in two dimensions. Each ellipse is a line of equal Mahalanobis distance around a class mean, and all classes share the same covariance, so the ellipses share their shape. An in-distribution input lies close to a class cluster, while an out-of-distribution input lies far from every cluster.

2.3 Feature-based detection with the Mahalanobis distance

A second family of methods looks inside the network instead of only at its output. Every layer of the network turns its input into a feature vector, which is an intermediate numerical representation [5]. When the network is trained well, inputs from the same class tend to have similar feature vectors, forming one cluster for each class in the feature space. An input that falls far from every cluster matches no known class, meaning it is likely out-of-distribution. Figure 1 shows this idea in two dimensions: the ID input sits inside a class cluster, while the OOD input lies outside all of them.

Lee et al. turn this idea into a detector, and our work builds on their method [5]. They fit one cluster per class to the training feature vectors and score a new input by its distance to the nearest cluster. The distance they use is the Mahalanobis distance, a classical measure of the distance from a point to a cluster that accounts for the cluster’s shape [8]. A small distance means the input is close to a known class and is treated as in-distribution, while a large distance flags it as out-of-distribution. Across their experiments, this feature-based score detects OOD inputs better than both the softmax baseline and ODIN [5]. We define the score formally in Section 3.2.

The same score can be computed at any layer, not only the last one. Lee et al. find that earlier layers sometimes separate OOD inputs that the final layer misses. They also experiment with combining per-layer scores into a single score, with the combination of weights set so that uninformative layers count for less [5].

Which layers separate the classes best, and whether combining them beats the best single layer, are the two questions this project studies.

The difficulty of detection depends on how far the out-of-distribution inputs lie from the training data. Far-OOD inputs are a clearly different kind of data, such as photographs of objects shown to a model trained on handwritten digits. Near-OOD inputs are the same kind of data from classes the model never saw in training, such as new animal breeds shown to a model trained on other animal breeds [11, 13]. The Mahalanobis distance-based detector works well on far-OOD inputs but loses accuracy on near-OOD inputs. This is because the score sums contributions from all feature dimensions, including ones that do not differ between classes (e.g. background statistics shared by all traffic signs). For far-OOD inputs these non-discriminative dimensions still differ sharply from the training distribution and contribute useful signal, but for near-OOD inputs they remain similar to the training distribution, diluting the comparatively small signal carried by the dimensions that do separate the classes [11]. This project tests the detector in both settings.

2.4 Detection as a defense against model cloning

We now turn to the security risk. An attacker can copy a model that is behind a query interface. In a model extraction attack, an adversary with only black-box access sends many inputs to the model, observes the returned predictions, and uses them to reconstruct an equivalent model [12]. Earlier work reverse-engineered simpler classifiers from their outputs in the same way [7]. Tramèr et al. show that such attacks can copy commercial models with near-perfect fidelity [12].

These attacks connect to out-of-distribution detection because their queries tend to be OOD. An attacker rarely has the model’s training data, so it queries with whatever inputs it has at hand. Orekondy et al. show that querying with images from a different distribution than the training data is enough to train a good copy [10]. Kariyappa and Qureshi confirm this pattern, namely, for known stealing attacks a large fraction of the attacker’s queries are out-of-distribution [4]. Figure 2 shows the effect on the maximum softmax probability, where ordinary user queries produce high scores, while attacker queries produce low scores, showing that the score alone separates the two. A detector that flags OOD inputs can therefore also flag a cloning attempt. The Mahalanobis detector we study addresses both risks from Section 2.1: it protects against unsafe predictions on unfamiliar inputs (safety), and it helps detect model cloning (security).

3 Methodology

3.1 Baseline

We compare all Mahalanobis-based detectors against the maximum softmax probability (MSP) baseline; see Section 2.2

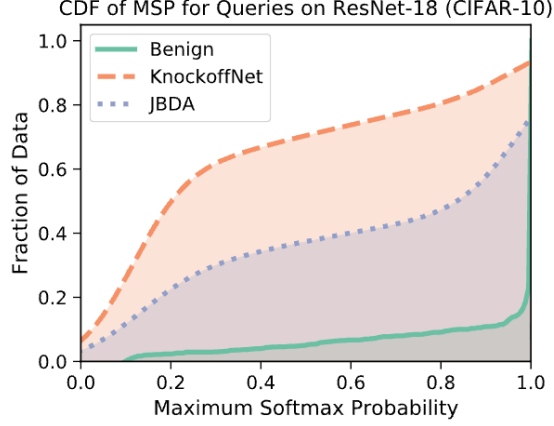


Fig. 2. Cumulative distribution of the maximum softmax probability for queries to a ResNet-18 classifier trained on CIFAR-10. Benign user queries concentrate at high softmax probability, which marks them as in-distribution. Queries from two model-stealing attacks (KnockoffNet and JBDA) place much of their mass at low softmax probability, which marks them as out-of-distribution. Reproduced from [4].

3.2 Mahalanobis distance

We detect out-of-distribution inputs with the Mahalanobis distance score of Lee et al. [5], computed from the features of our fine-tuned classifier. This section defines the score for a single layer and additionally its combination across layers.

Per-class Gaussians. Let $f(x)$ be the feature vector that a chosen layer produces for an input x . On the training set, we fit one Gaussian per class to these feature vectors. Each class c has its own mean $\hat{\mu}_c$ (center), while all classes share one covariance matrix $\hat{\Sigma}$ (a tied covariance) which gives all the class clusters the same shape. The parameters are estimated from the training features:

$$\hat{\mu}_c = \frac{1}{N_c} \sum_{i: y_i=c} f(x_i), \quad \hat{\Sigma} = \frac{1}{N} \sum_c \sum_{i: y_i=c} (f(x_i) - \hat{\mu}_c)(f(x_i) - \hat{\mu}_c)^\top, \quad (2)$$

where N_c is the number of training inputs of class c and N is the total number of training inputs. Sharing one covariance across classes avoids unreliable per-class estimates when some classes have few training samples. At layers with many feature dimensions the estimated covariance can be close to singular, which makes its inverse unstable, so we shrink the covariance toward a diagonal matrix before inverting it.

Single-layer score. For a new input x , we compute its Mahalanobis distance to each class and keep the smallest. The score is the negative of that distance:

$$M(x) = -\min_c (f(x) - \hat{\mu}_c)^\top \hat{\Sigma}^{-1} (f(x) - \hat{\mu}_c). \quad (3)$$

A high $M(x)$ means the input lies close to some class and is treated as in-distribution. A low $M(x)$ means the input lies far from every class and is flagged as out-of-distribution.

Combining layers. Equations 2 and 3 apply to any layer. In practice, we compute a separate score M_ℓ at each layer ℓ and also combine the per-layer scores into one:

$$M_{\text{comb}}(x) = \sum_{\ell} \alpha_{\ell} M_{\ell}(x). \quad (4)$$

We use three combiners that differ in how the weights α_{ℓ} are chosen. The first two use fixed weights that treat every layer equally. Namely, we first standardize each layer’s score with the in-distribution training statistics (subtract the mean and divide by the standard deviation) and then take either the sum of these standardized scores ($\alpha_{\ell} = 1$) or their mean ($\alpha_{\ell} = 1/L$ for L layers). The third combiner learns the weights with logistic regression directly from the raw per-layer scores. This allows the learned weights to absorb the scale differences, in addition to uninformative layers receiving a weight near zero. We tune the weights by separating clean training inputs from slightly perturbed copies of those same inputs [5]. The perturbation comes from the Fast Gradient Sign Method (FGSM), which shifts an input by a small step in the direction that most increases the classifier’s loss [1]. These perturbed inputs replace actual unfamiliar inputs, so no real OOD data is needed. The specific schemes are described in Section 4.4.

4 Experimental Setup

4.1 Model

For this research, we fine-tuned MobileNetV2 (pre-trained on ImageNet) on each in-distribution (ID) dataset by replacing the final classification head with a linear layer sized to the dataset’s number of classes. Our training uses standard cross-entropy loss, AdamW as the optimizer to ensure generalization and stability, and a cosine-annealing LR scheduler for extra stability. All OOD detection is performed post-hoc on the frozen fine-tuned ID model; the model has never seen OOD data to ensure no leakage. The logistic regression fusion variant is the only exception that uses auxiliary tuning data as described in Section 4.4. To assess whether the observed detection patterns are specific to MobileNetV2 or generalize across architectures, we additionally fine-tune MobileNetV1 on the same datasets under identical training conditions.

4.2 Datasets

Three experimental setups are evaluated, each varying either in-distribution data or the nature of the OOD split.

Experiment 1 — Far-OOD. The model is trained on GTSRB (43 traffic sign classes). During test time, held-out GTSRB test images serve as ID samples and Tiny-ImageNet (natural images) or Oxford-IIIT Pet (37 cat and dog breeds) serve as OOD. These different types of images differ substantially in texture, colour and semantic content, making this a far-OOD environment.

Experiment 2 — Near-OOD. The model is trained on a subset of 38 GTSRB classes; 5 classes are withheld to function as OOD at test time. The split needs to satisfy two constraints: (1) Held-out classes must have enough samples for a useful OOD test pool, (2) held-out classes must not be visually idiosyncratic: if the only octagonal sign is held out, the detector exploits shape alone and the result would overstate true near-OOD performance. Because the held-out signs share shape and colour statistics with the training classes, the detector cannot rely on low-level appearance differences and must exploit the learned feature geometry.

Experiment 3 — Harder-ID. The model is trained on the Oxford-IIIT Pet dataset. OOD is again Tiny-ImageNet, with all cat and dog classes removed to prevent destructive overlap. This experiment tests whether our OOD detection approaches also work effectively on less visually homogeneous ID data.

4.3 Feature Extraction

MobileNetV2 produces 19 intermediate feature tensors, one per MBConv block (blocks 1-17) plus the stem convolution and the final 1×1 convolution (block 18). Each block outputs a 4D tensor of shape (B, C, H, W) . In order to obtain a fixed length feature vector suitable for fitting a Gaussian, we apply adaptive average pooling with grid size $k \times k$ before flattening:

- **Early blocks (0-6) (large spatial resolution, few channels):** $k = 4$, retaining some spatial structure.
- **Mid blocks (7-13):** $k = 2$
- **Deep blocks (14-18) (small spatial resolution, many channels):** $k = 1$ (global average pooling)

This process yields feature vectors of length $C \cdot k^2$ per block, with early blocks producing 512 to 1280 at block 18.

4.4 OOD Detection Methods

MSP baseline. The maximum softmax probability of the classifier is used as the ID-OOD confidence score, as proposed by [3]. Lower MSP indicates lower classification confidence and potential OOD.

Mahalanobis distance (single layer). For each candidate layer, we fit a class-conditional Gaussian with tied covariance on the fine-tuned model’s training set features; Section 3.2. The layer score $M(x) = -\min_c D_c(x)$ where $D_c(x)$ is the Mahalanobis distance to class c , computed independently for each layer. We evaluate each of the 19 single-layer OOD-detectors separately by several metrics

discussed in Section 4.5.

Multi-layer fusion. Per-layer scores are z-normalised and combined using three schemes: (a) uniform sum of z-normalised per-layer scores, (b) mean of z-normalised scores, and (c) logistic regression over the per-layer score vector. For the logistic regression variant, weights are trained on clean ID images versus FGSM-perturbed ID images (Section 3.2); no real OOD data is used, following the tuning protocol of [5].

4.5 Evaluation Metrics

All methods are evaluated on three metrics, (except the excluded AUPR metric) matching [5]:

- **AUROC**: Threshold-free measure of separation; 1.0 is perfect, 0.5 is random chance.
- **TNR@95**: True negative rate (fraction of OOD inputs correctly rejected) at the threshold that accepts 95% of in-distribution inputs.
- **Detection accuracy (DetAcc)**: The best balanced accuracy achievable over all thresholds: $\max_t [0.5 \cdot (P(ID \geq t) + P(OOD < t))]$.

5 Results and Analysis

Table 1. Classification Accuracy of trained models used for the results.

Model	Classes	Val	Test
MobileNetV2 – GTSRB (full)	43	0.9996	0.9925
MobileNetV2 – GTSRB (kept)	38	0.9990	0.9910
MobileNetV2 – Oxford Pet	37	0.8862	0.8607

The models used in testing Mahalanobis distance or any of the data presented below, were trained such that their accuracy on the validation dataset approximately matched the highest trained model accuracies we could find in the literature.

In both Far-OOD scenarios, both the MSP baseline and the Mahalanobis distance reach comparable performances with AUROC scores greater than 0.995 (Table 2). This great performance reflects the large visual gap between the in-distribution and out-of-distribution data: traffic signs differ strongly from natural ImageNet images or Oxford Pet images that even softmax baseline separates them with high confidence. Such that even the baseline MSP is able to quite confidently separate the data.

However, even here, the Mahalanobis distance outperforms the MSP baseline at the highest layers, and we can see that Mahalanobis distance is most effective in

Table 2. Selected per-layer metrics. ID: GTSRB (43-class). Full per-layer tables can be found in the appendix.

OOD input	Layer	AUROC	TNR@95	DetAcc
Tiny-ImageNet	block0	0.904	0.541	0.818
	block2	0.989	0.975	0.964
	block16	0.998	0.999	0.987
	MSP baseline	0.995	0.998	0.981
Oxford Pets	block0	0.910	0.501	0.831
	block2	1.000	1.000	0.992
	block16	0.998	1.000	0.987
	MSP baseline	0.996	1.000	0.984

Table 3. Selected per-layer metrics. ID: GTSRB 38-class model. OOD: 5 held-out GTSRB classes.

Layer	AUROC	TNR@95	DetAcc
block0	0.553	0.084	0.547
block3	0.571	0.081	0.549
block6	0.621	0.149	0.586
block13	0.696	0.212	0.638
block16	0.941	0.617	0.887
block17	0.963	0.689	0.916
block18	0.955	0.657	0.898
MSP baseline	0.975	0.867	0.933

the deeper layers, confirming that these layers contain more class-discriminative features.

Upon further testing of the Mahalanobis distance effectiveness between OOD data that is similar to the ID data, the AUROC scores are similar at deeper layers. However, the MSP baseline is marginally higher (Table 3). This is due to the fact that the accuracy of this model is higher than 99%, making any out-of-distribution data apparent due to the very high confidence of trained on data. Still even here Mahalanobis distance is able to clearly differentiate between in-distribution and out-of-distribution data. Following from this, we investigated whether this effectiveness transfers to a model that is less accurate and therefore has a wider range of confidence scores, making the use of MSP for OOD detection less effective.

To investigate this, we trained a model on the Oxford Pet dataset that classified 37 different breeds of dogs and cats. This type of classification is inherently less confident, and we achieved an accuracy of 85%. The MSP baseline received a significantly worse score of 0.85, indicating a decreased OOD differentiation in the less-accurate models (Table 4). In this experiment we measured an increased effectiveness of Mahalanobis distance compared to the baseline. As Mahalanobis distance effectively differentiates between OOD and ID data much more effec-

Table 4. Selected per-layer metrics. ID: Oxford Pet (37-class). OOD: Tiny-ImageNet (cat/dog ids excluded).

Layer	AUROC	TNR@95	DetAcc
block1	0.492	0.086	0.520
block6	0.607	0.055	0.582
block10	0.959	0.795	0.904
block14	0.987	0.936	0.950
block17	0.992	0.980	0.966
block18	0.889	0.413	0.811
MSP baseline	0.850	0.392	0.770

tively than the baseline, even achieving an AUROC score of 0.992 at the highest layers, a near perfect differentiation.

Table 5. Multi-layer combination methods across OOD scenarios. Best single-layer result shown for reference.

Setting	Method	AUROC	TNR@95
GTSRB vs. Tiny-ImageNet	Best single (block16)	0.998	0.999
	mean _z (all)	0.999	1.000
	mean _z (custom top-5)	0.999	1.000
	logreg_fgsm (all)	0.978	0.921
	MSP baseline	0.995	0.998
GTSRB vs. GTSRB held-out	Best single (block17)	0.963	0.689
	mean _z (all)	0.767	0.273
	logreg_fgsm (custom top-2)	0.965	0.687
	MSP baseline	0.975	0.867
Oxford Pet vs. Tiny-ImageNet	Best single (block17)	0.992	0.980
	mean _z (all)	0.965	0.796
	logreg_fgsm (all)	0.994	0.989
	MSP baseline	0.850	0.392

We also investigated the effectiveness of combining Mahalanobis distance scores from multiple layers to achieve a better differentiation between OOD and ID data. Using the normalized Mean and normalized Sum of the Mahalanobis distance showed little to no improvement and even in some cases showed meaningfully decreased results compared to just purely using the best layer for differentiation. Logistic regression fusion showed small improvements in most settings, but the gains were marginal: at best a 0.002 increase in AUROC over the best single layer, and at worst a 0.021 decrease. This, however, could also be due to the fact that Mahalanobis distance was already able to differentiate

beyond the AUROC score of 0.96 for the best layer in every model, leaving little room for fusion to improve upon.

To find out whether these results were dependent on the specific model architecture, we additionally did analysis on the MobileNetV1 architecture. These models were trained to have accuracies comparable as those of the V2 model. The best-layer results were similar, that is, the AUROC score was highest at the last few layers, and thus it indicated that the Mahalanobis distance had the best OOD differentiation in the last few layers. Those were layers 13/12 in model V1 and 17/16 in model V2, as the V1 model had in total 13 layers and the V2 model had 17 layers.

Table 6. Peak per-layer Mahalanobis performance by architecture and scenario. Best layer shown in parentheses.

Scenario	Arch.	AUROC	TNR@95	DetAcc
GTSRB vs. Tiny-ImageNet	V2 (block16)	0.998	0.999	0.987
	V1 (block13)	0.991	0.999	0.977
GTSRB vs. Oxford Pets	V2 (block2)	1.000	1.000	0.992
	V1 (block13)	0.988	0.999	0.977
GTSRB near-OOD (held-out classes)	V2 (block17)	0.963	0.689	0.916
	V1 (block13)	0.938	0.567	0.874
Oxford Pet vs. Tiny-ImageNet	V2 (block17)	0.992	0.980	0.966
	V1 (block12)	0.969	0.844	0.911

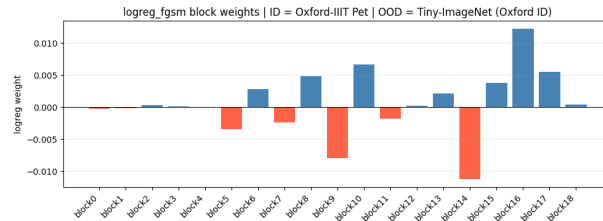


Fig. 3. Learned logistic regression weights per block for the Oxford Pet ID setting. Positive weights are blue, negative weights are orange.

Figure 3 shows the learned per-block weights of the logistic regression model. The weights show no consistent depth-dependent pattern: rather than increasing smoothly with depth as the single-layer AUROC results suggest, they jump in sign and magnitude between adjacent blocks. In the Oxford Pet ID setting, block 16 receives the largest positive weight, consistent with its strong single-layer AUROC of 0.992, yet block 14 receives a large negative weight despite also

scoring 0.987 individually. Since the weights are tuned on FGSM-transformed ID images (rather than real OOD data), this instability likely reflects overfitting to this small tuning set, which serves as a proxy for OOD data rather than containing actual OOD samples.

6 Discussion

In our experiments, for the datasets and architectures observed Far-OOD detection appears to be largely solved. In GTSRB vs. Oxford-IIIT Pet appears to saturate as early as block 2 (AUROC 1.000): in that both detectors perform very effectively (best Mahalanobis 0.998–1.000; MSP 0.995, Table 2). In our experiments behavior worth further examination occurs in the near-OOD as well as low-confidence-ID datasets and models.

Our results suggest that the most effective layer for differentiation is the last few layers, however not the last layer. Over-specialization to the trained dataset causes discrimination to peak at the last convolutional blocks (16–17) then regress at the final 1×1 projection (block 18: 0.992 $\rightarrow$ 0.889, Table 4) which is likely due to the fact that the last layer is optimized for class summarization rather than extracting features that Mahalanobis distance could use to differentiate. Near-OOD is similar to far-OOD: blocks from 0–13 are nearly random (0.55–0.70), with a signal only at 14–17 (Table 3). According to our experiments, a rule we could summarize that seems to hold for all tested architectures and datasets is to "use the last semantic block."

MSP based detection performs quite well in Far-OOD settings with approximately a 0.99 AUROC score and is comparable to single-layer Mahalanobis on near-OOD differentiation (0.975 vs. 0.963 AUROC; 0.867 vs. 0.689 TNR@95). However, we observed that the MSP based differentiation effectiveness collapses on lower accuracy models such as the Oxford Pet classifier which has an accuracy of 86%. The MSP baseline achieves the following AUROC and TNR@95 (0.850/0.392) whereas Mahalanobis maintains its differentiation effectiveness with the following scores (0.992 / 0.980).

Our classifiers accuracy appears to be a prerequisite for MSP baseline based OOD detection effectiveness. In our experiments the Mahalanobis distance based OOD detection seems to be able to maintain its effectiveness, despite the accuracy of the classifier.

In our experiments layer combination is either somewhat inferior to best layer choice or at best comparable as per AUROC score. No fusion strategy consistently succeeds: the FGSM-tuned logistic combiner is inconsistent (it is best on Oxford at 0.994, but gets 0.978 vs. 0.998 on far-OOD and trails MSP near-OOD). In our experiments, averaging z-scores only helps in a saturated far-OOD and

gets worse if used to differentiate near-OOD ($\text{mean-z} \rightarrow 0.767$).

The patterns we observed in our experiments appear to be independent of architecture. It appears that MobileNetV1 replicates our findings from MobileNetV2 (far-OOD 0.99; near-OOD 0.94 at the block 13/13 - and peak in the last blocks) despite the fact that it only has 13 layers appears to indicate that our finding regarding the use of the last semantic block for Mahalanobis distance differentiation holds.

7 Conclusion and Future Work

We compared Mahalanobis distance scoring against the maximum softmax probability baseline for out-of-distribution detection. We computed the score at every layer of multiple fine-tuned MobileNet classifiers and tested it across far-OOD and near-OOD splits, and on a lower-confidence classifier.

The accuracy of the Mahalanobis detector depended heavily on the classifier. When the classifier was almost always correct, the softmax baseline was already very strong. In such cases, the Mahalanobis score only improved slightly and sometimes even performed worse. When the classifier was more uncertain (i.e., the Oxford Pet model), the softmax score dropped substantially to 0.850 AU-ROC while the Mahalanobis score held at 0.992. This is because the classifier is unsure even on in-distribution inputs, meaning their confidence drop, and as a result the softmax score cannot separate them from OOD inputs. Whereas the Mahalanobis score measures distance in feature space, so low confidence does not affect it. That said, it is not entirely independent of classifier accuracy. A less accurate classifier also produces less well-separated feature clusters, which makes the Gaussian estimates noisier and reduces Mahalanobis separation in turn.

We also studied which layer is best to use in OOD detection and whether combining multiple layers resulted in a better score for use in Mahalanobis distance based OOD detection. Far-OOD was quite easy, and most layers separated the data almost perfectly. In the more difficult near-OOD and low-confidence settings, the layer mattered more. Generally, the best separation came from a deep layer, but never the final one, and earlier layers carried very little signal. Additionally, combining layers also resulted in little to no improvement. Thus, picking one good layer worked just as well as fusing many, if not better.

These patterns held across both tested MobileNet versions. MobileNetV1 and MobileNetV2 use different block designs, yet both showed the same behavior, which suggests the pattern does not depend on the exact network design.

However, a few limitations remain, each of which points to a next step. Firstly, we have seen that the Mahalanobis detector is weaker on near-OOD inputs, which happens because feature dimensions that look the same for every class outweigh the few that tell classes apart. A known fix for this removes the contribution of these dimensions within each layer's score and instead lets the few that are

able to separate the classes decide the score, resulting in the near-OOD detection improving [11]. Secondly, we only tested two backbones from one family (MobileNet), so other architectures and datasets would strengthen the findings. Then, we also only used a single tied covariance matrix, which could instead be done with a separate covariance per class. This would potentially capture class shapes that the shared one cannot. Finally, we assumed that flagging OOD inputs would allow us to catch model-cloning queries, but we did not test this against a real attack. Running the detector against an actual extraction attack would show whether it is able to catch an attacker’s queries in practice.

7.1 Contributions

Filip Atanasovski - 1833103. Wrote the Results and Analysis section, worked to redo the Discussion section, worked to fix the Introduction Section. On the code side I implemented Mahalanobis Distance scoring function that is used in the rest of the pipeline, trained the GTSRB model and evaluated ImageNet and created plots and data for MSP baseline and evaluated it for every architecture and dataset.

Thijmen Schinkelshoek - 1663305. On the code side, I did much of the project’s foundational implementation, in particular structuring the code into a reusable pipeline that the other experiments build on. I trained the final GTSRB classifier by adding general improvements to the training procedure. I also fully implemented the held-out near-OOD experiment and implemented the functionality for the multi-layer combination methods. For the report, I wrote the Background and Related Work section, the Mahalanobis distance methodology sections, and the Conclusion.

Matus Feltovic - 1718657. Built the entire work stream for the MobileNetV1 architecture, trained the model, compared V1 vs V2 on OOD detection comparison on GTSRB traffic signs (including evaluation runs for near-OOD and far-OOD and Oxford-as-Is invariant). This determined that V2 narrowly wins over V1. For the report - I wrote the Introduction and the Discussion sections.

Thijmen Adriaansen - 2220822. As for coding: improved model accuracy, implementation of the far-OOD experiment, and visualization of the logistic regression weight distribution. For the report: Methodology, Experimental Setup, and assisted in writing the results sections of the report. Also generally contributed to all sections of the report.

References

1. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples (2015), <https://arxiv.org/abs/1412.6572>
2. Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: Precup, D., Teh, Y.W. (eds.) Proceedings of the 34th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 70, pp. 1321–1330. PMLR (06–11 Aug 2017), <https://proceedings.mlr.press/v70/guo17a.html>
3. Hendrycks, D., Gimpel, K.: A baseline for detecting misclassified and out-of-distribution examples in neural networks (2018), <https://arxiv.org/abs/1610.02136>
4. Kariyappa, S., Qureshi, M.K.: Defending against model stealing attacks with adaptive misinformation (2019), <https://arxiv.org/abs/1911.07100>
5. Lee, K., Lee, K., Lee, H., Shin, J.: A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In: Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (eds.) Advances in Neural Information Processing Systems. vol. 31. Curran Associates, Inc. (2018), https://proceedings.neurips.cc/paper_files/paper/2018/file/abdeb6f575ac5c6676b747bca8d09cc2-Paper.pdf
6. Liang, S., Li, Y., Srikant, R.: Enhancing the reliability of out-of-distribution image detection in neural networks (2020), <https://arxiv.org/abs/1706.02690>
7. Lowd, D., Meek, C.: Adversarial learning. In: Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery in Data Mining. p. 641–647. KDD '05, Association for Computing Machinery, New York, NY, USA (2005). <https://doi.org/10.1145/1081870.1081950>, <https://doi.org/10.1145/1081870.1081950>
8. Mahalanobis, P.C.: On the generalised distance in statistics. Proceedings of the National Institute of Sciences of India **2**(1), 49–55 (1936)
9. Nguyen, A., Yosinski, J., Clune, J.: Deep neural networks are easily fooled: High confidence predictions for unrecognizable images (2015), <https://arxiv.org/abs/1412.1897>
10. Orekondy, T., Schiele, B., Fritz, M.: Knockoff nets: Stealing functionality of black-box models (2018), <https://arxiv.org/abs/1812.02766>
11. Ren, J., Fort, S., Liu, J., Roy, A.G., Padhy, S., Lakshminarayanan, B.: A simple fix to mahalanobis distance for improving near-ood detection (2021), <https://arxiv.org/abs/2106.09022>
12. Tramèr, F., Zhang, F., Juels, A., Reiter, M.K., Ristenpart, T.: Stealing machine learning models via prediction APIs. In: 25th USENIX Security Symposium (USENIX Security 16). pp. 601–618. USENIX Association, Austin, TX (Aug 2016), <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/tramer>
13. Yang, J., Zhou, K., Li, Y., Liu, Z.: Generalized out-of-distribution detection: A survey. International Journal of Computer Vision **132**(12), 5635–5662 (2024). <https://doi.org/10.1007/s11263-024-02117-4>, <https://doi.org/10.1007/s11263-024-02117-4>